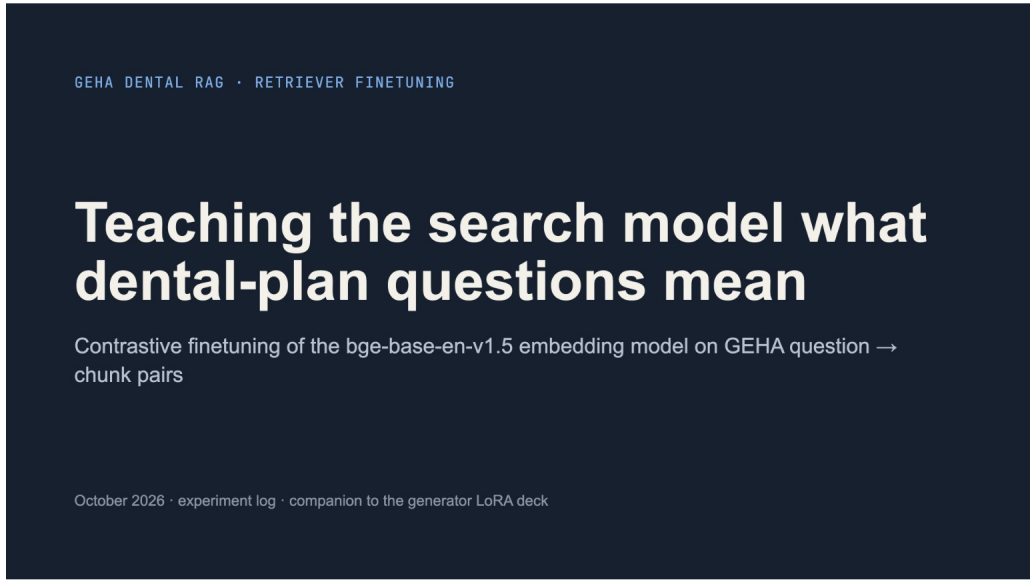
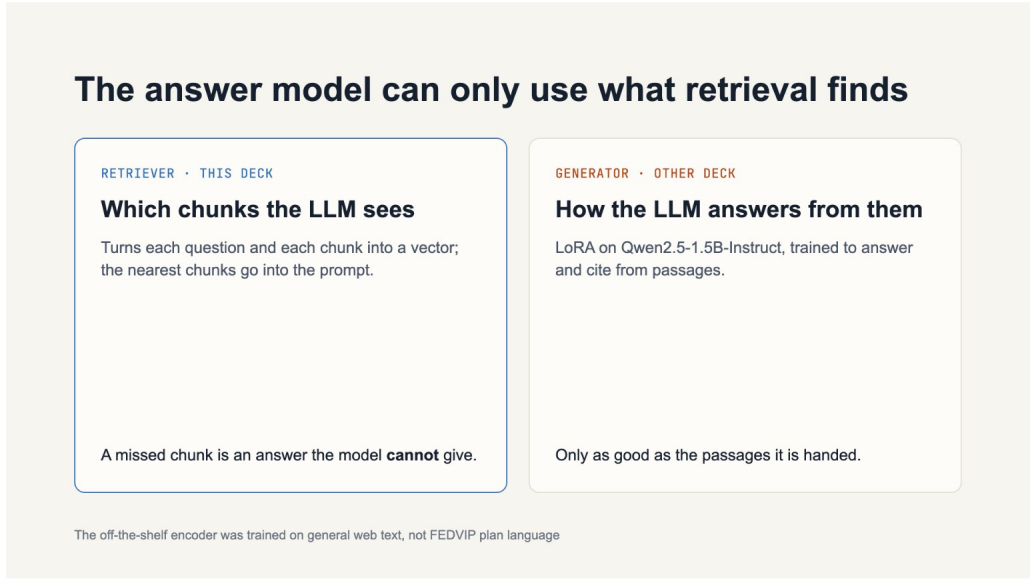
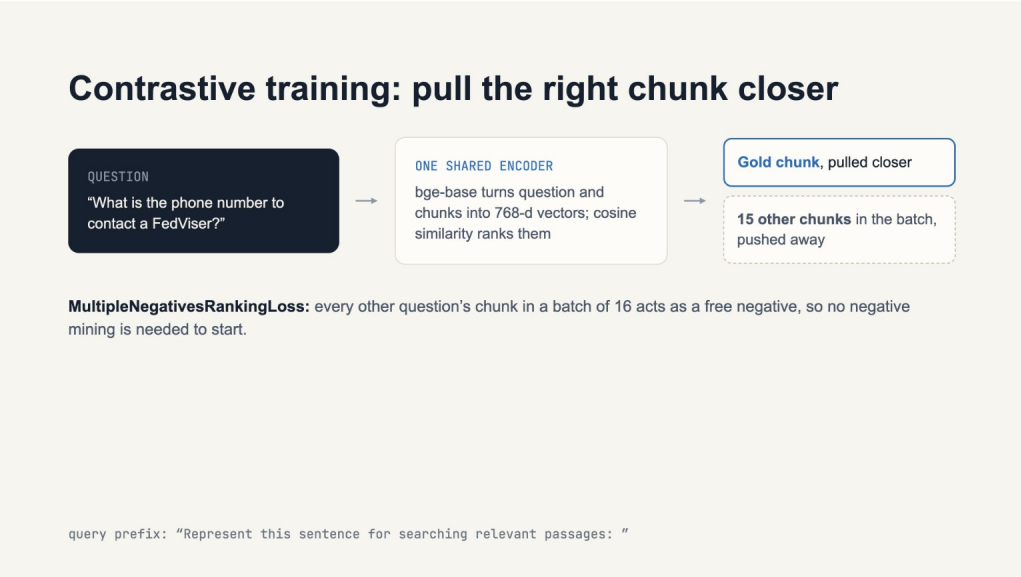


This deck covers the retriever half of RAG finetuning: why it matters, how contrastive training works, the data and splits, the training run, held-out results with confidence intervals, whether it overfits, and how it compares with the generator finetuning.



Retrieval errors cap answer quality: if the chunk holding the answer never reaches the prompt, no amount of generator tuning recovers it. That is why the retriever was finetuned first. The base model, bge-base-en-v1.5, is a strong general embedding model but has never seen dental-plan terms like FedViser, Self Plus One or premium rating areas.



Each training example is a question paired with the chunk that answers it. The loss treats a batch as a ranking problem: for each question, its own chunk should score highest among all 16 chunks in the batch. Over training, the encoder learns that GEHA questions and the policy text that answers them belong close together. The bge query prefix is kept for training and evaluation.

Data: synthetic questions, and a test set from another document

	train	validation	test
Document	guide	guide	brochure
Sections	22	6	72
Chunks	34	8	167
QA pairs	78	24	41
Question → chunk pairs	135	–	–
Search corpus at eval	–	42	167

How it was built

RAGAS + gpt-4.1-mini wrote questions from Docling chunks

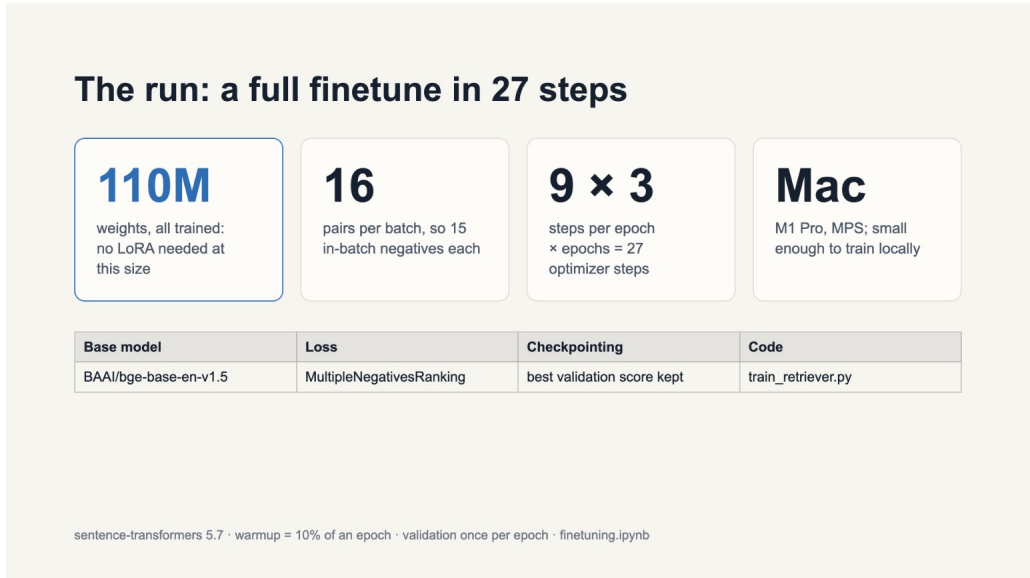
Table-of-contents chunks removed

Train and validation split by section

Test = the plan brochure, never trained on

Guide = 2026 GEHA dental benefits guide · Brochure = 2026 plan brochure · prepare_retriever_splits.py, generate_ragas_qa.py

A question can cite up to three source chunks, so 78 training questions give 135 question-to-chunk pairs. Validation searches the 42 guide chunks; the test searches all 167 brochure chunks, which is a much harder, more realistic setting. These are the same splits the generator experiment later reused.



At 110 million parameters the encoder is small enough to train every weight, unlike the 1.5-billion-parameter generator, which needed LoRA. 135 pairs in batches of 16 make 9 steps per epoch, 27 in total, and the run fit comfortably on the Mac. Query latency stayed the same: about 13 to 15 milliseconds median per question, since the architecture is unchanged.

Validation: most of the gain arrives in epoch 1

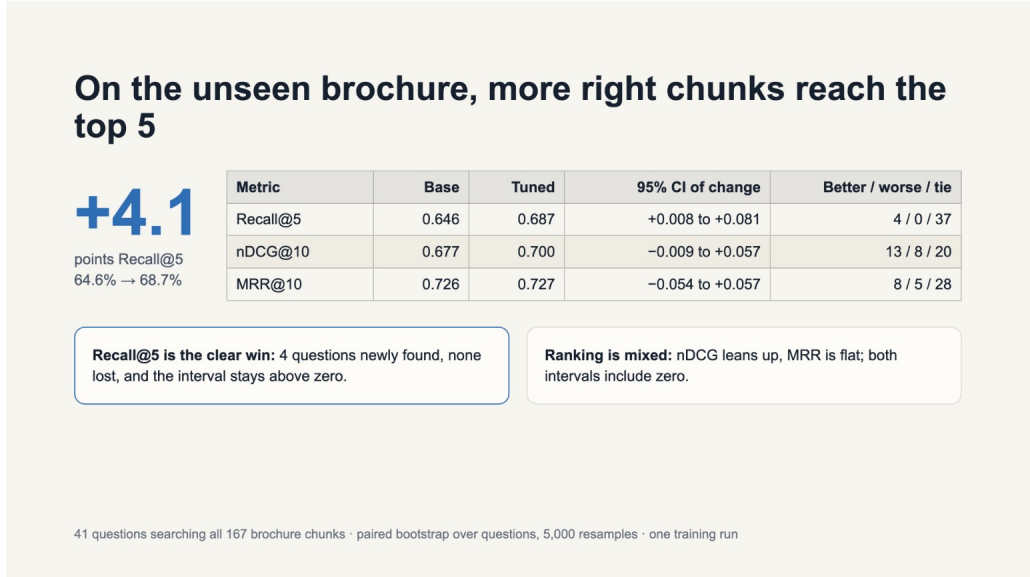
Validation (24 questions)	Before	Epoch 1	Epoch 2	Epoch 3
Recall@5	0.743	0.812	0.847	0.868
nDCG@10	0.754	0.822	0.822	0.818
MRR@10	0.832	0.896	0.878	0.878
Top-1 hit rate	–	0.833	0.792	0.792

Epoch 1: every ranking metric jumps; nDCG +0.07, MRR +0.06.

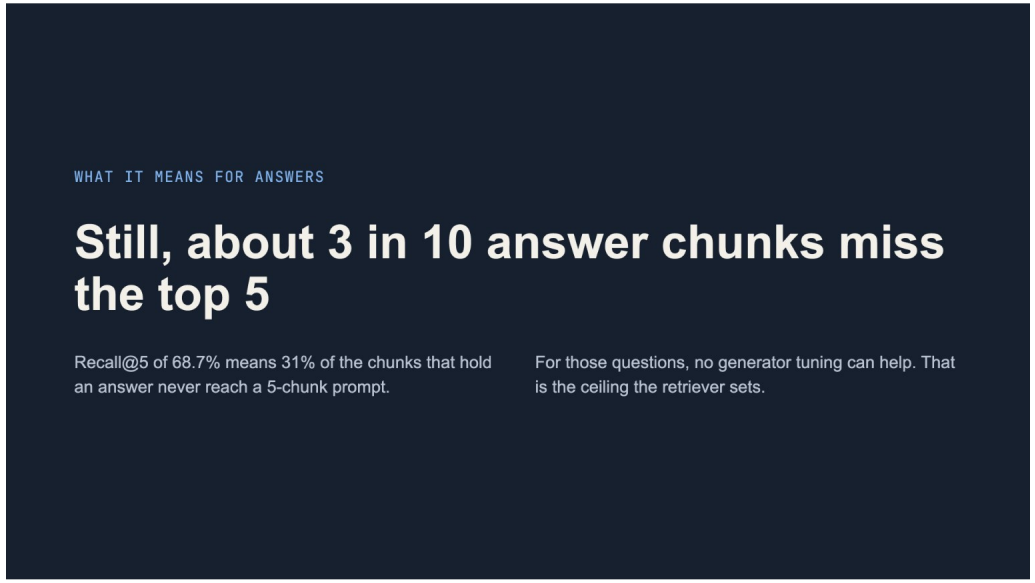
Epochs 2–3: recall keeps rising, but nDCG flattens and the top-1 hit slips.

Search corpus = 42 guide chunks · bold = best per row · from the evaluator's per-epoch CSV and the notebook

Validation runs once per epoch. Before training the base model already scores well on this small 42-chunk corpus. Epoch 1 brings the big jump on every metric. After that, recall at 5 keeps creeping up, but nDCG is flat and the top-ranked hit gets slightly worse: the model is getting more relevant chunks into the top five while getting a little less sure about the very first one.



This is the test that matters: a different document with a 167-chunk search space. Recall at 5 went from 64.6 to 68.7 percent. Four questions whose answer chunk was missing from the top five now find it, and no question lost its chunk, so that gain is solid even at this small size. The ordering metrics are less clear: nDCG improves on average but helps 13 questions and hurts 8, and MRR barely moves. I recomputed every per-question score from the saved model; the averages match results/comparison.json exactly.



The finetune moved recall at 5 from about 65 to about 69 percent on the unseen brochure. That is real progress, but it also means roughly three out of ten answer chunks are still not retrieved into a five-chunk context. This is the biggest remaining lever for end-to-end answer quality, bigger than anything the generator can do.



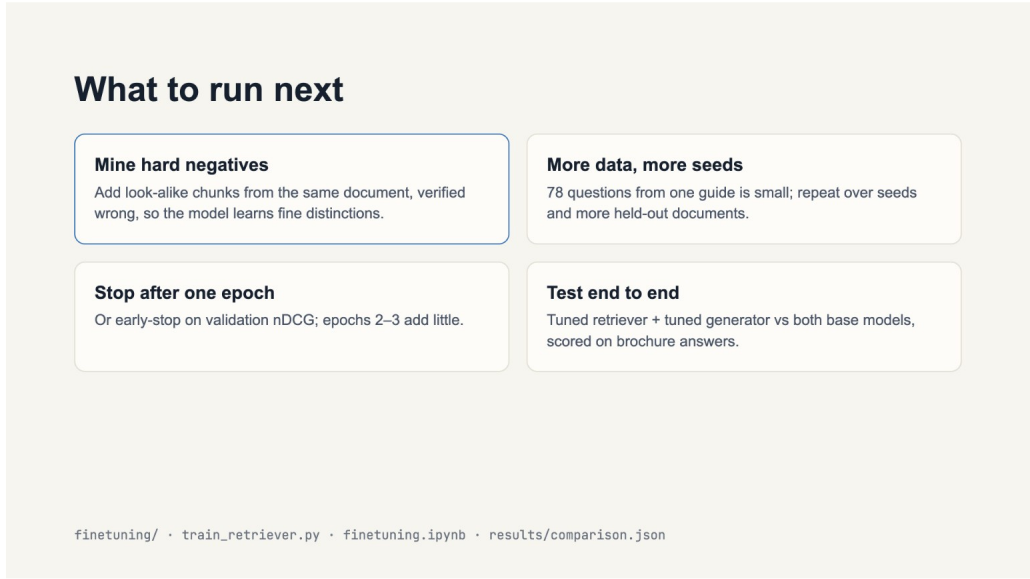
Short answer: no clear overfitting. Recall keeps rising on validation and the model still generalizes to a different document. But beyond epoch 1 the ranking quality stops improving, and the small dip in the top-ranked hit is a single question out of 24, so it is within noise. With this little data, training one epoch, or adding early stopping on nDCG, would give the same result for a third of the compute.

Retriever and generator finetuning side by side

	Retriever (this deck)	Generator
Model	bge-base-en-v1.5, 110M	Qwen2.5-1.5B-Instruct, 1.5B
Method	Full finetune, contrastive	LoRA r=16, 18.5M trainable
Training data	135 question → chunk pairs	156 chat examples, direct + CoT
Steps	27; gain mostly in epoch 1	60; best at step 20, epoch 1
Hardware	M1 Pro Mac	Colab A100 (Mac ≈47× slower)
Held-out gain	Recall@5 +4.1 points	Answer F1 +0.06, gold citations +0.12
After epoch 1	Plateau	Clear overfitting

Both evaluated on the same 41 brochure questions · one training run each

Both finetunes used the same synthetic questions and the same splits, and both were judged on the same unseen brochure. The retriever is small enough for a full finetune on a laptop; the generator needed LoRA and a GPU. Both got most of their benefit in the first epoch. Neither has yet been tested together in one end-to-end pipeline.



Four next steps. Hard negatives are the standard way to push retrieval further, and the finetuning README already calls for verified ones before the next round. More questions and repeated seeds will tell us how stable the 4-point gain is. Training one epoch saves compute with no loss. And the real goal is end to end: running both finetuned models together and checking whether answers on the brochure actually improve.